Practice Sheet Full-Stack.

Priyanshu Sharma CU24025098O (B) (47)

① display the name and city all students older than 20, Sorted by name.

```
SELECT name, city
FROM Students
WHERE age > 20
ORDER BY name ASC;
```

OUTPUT →

name	city
Aarav	Jaipur
Karan	Delhi
Rohan	Jaipur.

② Find all student whose name starts with 'A'.

```
SELECT *
FROM STudents
WHERE name LIKE 'A%';
```

③ Display the 3 courses with the highest credits.

```
SELECT *
FROM Courses
ORDER BY credits DESC
 LIMIT 3;
```

④ Count how many students live in each city.

```
SELECT city, COUNT(*) AS Student-Count
  FROM Students.
  GROUP by city;
```

⑤ Find the average mark for each Course:

```
Select Course-id, AVG(marks) AS average-marks
  FROM enrollments
  GROUP BY Course-id
  HAVING AVG(marks) > 60;
```

⑥ INNER JOIN - Student Name, Course name and marks.

```
SELECT S.name, C.Course-name, e.marks.
FROM Students S
```

```
INNER JOIN enrollments e
  ON s.student_id = e.student_id
  INNER JOIN course c
    ON e.course_id = c.course_id;
```

⑦ LEFT JOIN - students who are not enrollment in any course

```
SELECT s.name
FROM students s
LEFT JOIN enrollment e
    ON s.student_Id = e.student_id
WHERE e.student_id IS NULL;
```

⑧ Subquery - enrollments having marks above the average

```
SELECT student_id
FROM enrollments
WHERE marks > (
   SELECT AVG(marks)
   FROM enrollments
);
```

⑨ Update and delete statements.

① Increase SQL Basics credits by 1

```
UPDATE courses
SET credits = credits + 1
WHERE course_name = 'SQL Basics';
```

② Delete enrollments where marks are below 35

```
DELETE FROM enrollments
  WHERE marks < 35;
```

⑩ CREATE TABLE - TEACHER.

```
CREATE TABLE student_TEACHER (
    teacher_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NU
    email VARCHAR(100) UNIQUE
    Salary DECIMAL(10,2) CHECK(
    course_id INT
    FOREIGN KEY (course_id) REF
    courses(course_id));
```